

BhoomiSetu

Team Handbook

Everything in this project, explained from zero: what the product does, how to run and demo it, every technology we used and why we chose it over the alternatives, every formula in the rules engine, the real numbers, the honest limitations, and the questions the judges will ask.

If you read only one paragraph, read this one.

BhoomiSetu puts all seven statutory stages of land acquisition onto one shared record that officials and landowners both read. A parcel cannot move to the next stage unless that stage's required documents are *verified* and no objection is open — the rule is enforced in code, not in a circular. Every parcel carries a 0–100 risk score built from four measurable signals, so the officer sees which parcels are about to stall *before* they stall. And every stage-history entry is SHA-256 hash-chained to the one before it, so a back-dated edit visibly breaks the chain.

PROBLEM STATEMENT

26016 — Real-Time National Land Acquisition & Management System

THEME / CATEGORY

Smart Automation · Software

ORGANISATION

Ministry of Rural Development · Dept. of Land Resources

HANDBOOK COVERS

Codebase as of the current build · 42 parcels, 5 projects, 5 states

Contents

- 1 **How to use this handbook** — who reads what, and the three sentences everyone must know
- 2 **The problem, properly understood** — what land acquisition is, the seven stages, why it goes wrong
- 3 **What we built** — two portals, five roles, the whole feature list
- 4 **Running it — the operations manual** — install, run, every screen, every control, troubleshooting
- 5 **The demo script** — exact clicks, exact numbers to expect, timing
- 6 **The tech stack — every choice and why** — what it is, why it beat the alternatives, what we deliberately did not use
- 7 **Architecture** — the five layers, the repository pattern, the file map
- 8 **The domain model** — every entity and field in the system
- 9 **The rules engine, formula by formula** — the gate, the statuses, the risk score, the hash chain, the scoping
- 10 **The real numbers** — figures computed from the actual dataset; memorise these
- 11 **Honest limitations** — what is real, what is simulated, what we must never claim
- 12 **The judge Q&A bank** — 40 questions with answers you can actually defend
- 13 **Pitch scripts** — 30 seconds, 2 minutes, 5 minutes
- 14 **Who prepares what** — splitting the handbook across the team
- 15 **Glossary** — every term, acronym and section number used in the pitch

A NOTE ON HONESTY

This handbook deliberately separates **what the code actually does** from **what a production deployment would do**. Chapter 11 lists every place the prototype simulates something. Learn that chapter as carefully as the others. A team that says “that part is simulated, here is exactly what would replace it” reads as competent. A team caught overclaiming loses the room instantly — and SIH judges are usually domain people who *will* catch it.

How to use this handbook

This document exists so that **any one of you can answer any question** a judge asks, not just the person who wrote that part of the code. SIH judging is a conversation, not a presentation — they will interrupt, they will pick one teammate and ask them something, and a team where only one person can explain the system reads as one person's project.

The minimum everybody must know cold

- 1. What it is.** "BhoomiSetu is a single shared record for a land parcel that carries it through all seven statutory stages of acquisition under the RFCTLARR Act of 2013 — with its documents, objections, compensation and delay risk attached to that same record. Officials and landowners read the same record."
- 2. What makes it different.** "Most tracking systems have a status field someone types into. Ours has a gate: a parcel physically cannot advance unless the stage's required documents are verified and no objection is open. And it does not just report delay — it predicts it, with an explainable 0–100 score."
- 3. Why it can actually be deployed.** "The whole data layer sits behind one interface with two implementations. The demo runs in memory with no backend; the same code runs against PostgreSQL. A state can point that interface at their existing DILRMP or Bhoomi Rashi records without touching the rules layer."

How to read it, by how much time you have

YOU HAVE	READ
15 minutes	Chapter 1 (this), Chapter 10 (the numbers), Chapter 13 (the 2-minute script).
1 hour	Add Chapter 2 (the domain), Chapter 5 (the demo script), Chapter 11 (limitations).
An evening	All of it. Then have a teammate quiz you from Chapter 12.
You are demoing	Chapters 4 and 5, until you can run the demo without looking.
You are answering tech questions	Chapters 6, 7, 9 — especially the "why not X" boxes.

THE ONE MISTAKE THAT KILLS HACKATHON TEAMS

Saying "we use AI" when you do not, or "it's secure" when authentication is out of scope. **We have no ML model anywhere in this project** — the risk score is deterministic arithmetic, and the document check is a size/filename heuristic. Both are labelled as such in the UI. That is a strength: an officer can verify a score by hand and a department can audit it. Say it that way, proudly, and you will never be caught out.

The problem, properly understood

2.1 What land acquisition actually is

When the government needs private land for a highway, a railway line, a canal or a port, it cannot simply take it. The **Right to Fair Compensation and Transparency in Land Acquisition, Rehabilitation and Resettlement Act, 2013** — everyone shortens it to **RFCTLARR** — lays down a sequence of steps, each with its own notice, its own paperwork, its own hearing rights and its own officer. Land is acquired one **parcel** at a time, and a parcel is identified by its **survey number** (in our data, things like `124/7` or `88/2`).

Roughly, the chain runs: the government publishes a notification of intent → officers go and physically survey the land jointly with the owner → the owner gets a window to object → the land and its assets are valued → compensation is approved → an *award* is formally declared → and finally the state takes physical possession. Separately, families who are displaced are entitled to **Rehabilitation and Resettlement (R&R)** benefits, tracked as its own workstream.

2.2 The seven stages we model

These are not invented. They are the sequence the code implements, with the day thresholds and required documents that live in `src/domain/constants.ts`. Learn this table — a judge from the Department of Land Resources will know it better than you do, and will notice if you don't.

#	STAGE	SLA	REQUIRED DOCUMENTS	RESPONSIBLE OFFICER
1	Notification Sec. 11 preliminary notification	30 days	Section 11 notification	Land Acquisition Officer
2	Joint Survey	21 days	Joint survey sketch Ownership record extract	Survey Officer
3	Objection Review Sec. 15 objections	30 days	Objection hearing minutes	Land Acquisition Officer
4	Valuation	21 days	Valuation report	Valuation Officer
5	Compensation Approval	21 days	Compensation approval statement	Compensation Officer
6	Award Sec. 23 award	21 days	Award order	District Collector
7	Possession Handover Sec. 38 possession	30 days	Possession handover memo	District Collector

WHY THE SLA NUMBERS MATTER

If a parcel sits in a stage for *more* days than that stage's threshold, the code marks it **stuck**. That single comparison — `daysInStage > thresholdDays` — is what turns a passive register into a monitoring system. Everything else (alerts, the risk score, the action queue) is built on top of it.

2.3 Why it goes wrong today

Five failures, and our five answers. This is the spine of the pitch.

THE FAILURE

WHAT BHOOMISETU DOES ABOUT IT

Seven stages, seven registers. Each stage is handled by a different officer keeping their own record. Nobody holds the whole chain for one parcel at once.	One parcel record carries stage, full history, documents, objections, compensation, owner and map coordinates together.
The centre cannot see the districts. To learn national status, the Ministry asks states for a “return” — a manually compiled report that is stale the day it arrives.	The National Dashboard computes the rollup live from the same parcel records the district officer is editing. No returns, no compilation.
The landowner is outside the system. There is no way to check where your survey number stands or why it stopped, short of visiting an office.	A zero-login portal: type your survey number, see your stage, your documents, what is blocking you, and file an objection — in Hindi or English, by voice if you cannot type.
Delay is discovered after the fact, at a project review, not on the day a parcel starts slipping.	Every parcel scores 0–100 for delay risk on four measurable signals. The Action Center is permanently sorted worst-first, with the responsible officer named.
Money is tracked apart from status. “How much compensation is still unpaid, and on which parcels?” is a manual exercise.	Compensation assessed vs paid is a field on the parcel, rolled up per project and nationally alongside area and R&R progress.

2.4 What the problem statement asks for, and where we answer it

PS 26016 ASKS FOR	WHERE IT LIVES IN BHOOMISETU
Real-time monitoring of acquisition status	District dashboard + National dashboard, both computed live from parcel records
End-to-end digital coverage of the process	All seven stages, with documents and objections at each
Decision support	Risk engine + Action Center + printable Reports with stage-duration-vs-SLA analysis
National-level view across projects and states	National Dashboard: area, compensation, timeline status, R&R, per project and per state
R&R progress	Its own summary card and its own per-project table on the National Dashboard
Transparency for the affected citizen	The landowner portal — and it needs no login at all

What we built

BhoomiSetu is a single React web application with **two portals** behind one landing page. Which portal you land in depends on the role you pick.

PORTAL ONE — THE OFFICIAL SIDE

Requires picking a role. Four official roles, each scoped differently. Contains the dashboards, the map, the parcel workspace, the risk queue and the reports.

`/official` , `/official/national` ,
`/official/parcel/:id` , `/official/action-center` , `/official/reports`

PORTAL TWO — THE LANDOWNER SIDE

No login at all. Deliberately anonymous. Search a survey number, read your own status, file an objection, hear it read aloud.

`/landowner` , `/landowner/status/:id`

3.1 The five roles and what each one sees

Scoping is real, and it is enforced in the data layer (`src/domain/access.ts`), not just by hiding menu items. A district officer cannot load another district's parcel even by pasting the URL — the parcel page re-checks scope before it renders.

ROLE	SCOPE REQUIRED AT SIGN-IN	LANDS ON	SEES
National Admin	None	National Dashboard	Everything: all 5 projects, all 5 states, all 42 parcels.
State Authority	State	National Dashboard	Only their state's projects and parcels. Maharashtra → 1 project, 25 parcels.
District Officer	State + District	District Dashboard	Only their district's parcels. Maharashtra/Pune → 6 parcels. Blocked from the national rollup.
Field Officer	State + District	District Dashboard	Same district scope as the District Officer. Blocked from the national rollup.
Landowner	None	Landowner search	Only the one parcel they look up, and only the citizen-facing fields.

THE DETAIL THAT IMPRESSES JUDGES

Try it live: sign in as District Officer for Maharashtra / Pune, then paste the URL of an Amravati parcel. You get **Access Restricted**, not the parcel. That is because `ParcelDetailPage` calls `isParcelInScope()` before it sets the parcel into state — scope is a data-layer decision, not a UI decision. Most hackathon “role-based access” is a hidden nav link; this one isn't.

3.2 The full feature list

FEATURE	WHAT IT DOES
---------	--------------

Seven-stage gated workflow	Advance is blocked unless required documents are <i>verified</i> and (in Objection Review) no objection is open.
District dashboard	Summary tiles, count-by-stage grid, six filters, an interactive map, an attention queue and the full parcel table.
National dashboard	Projects/states, area acquired vs notified, compensation paid vs assessed, timeline status, R&R rollup, a Gantt-style project timeline, and two tables.
Parcel workspace	Overview, workflow status, risk breakdown, seven-step stepper, advance form, audit ledger, document upload, document table with verify/reject, objection table with status control.
Risk engine & Action Center	Deterministic 0–100 score per parcel, four contributors shown individually, four risk bands, responsible officer named, next action written out. Sorted worst-first.
Tamper-evident audit ledger	SHA-256 hash chain over stage history, with a “Simulate tamper” button that visibly breaks the chain and a Reset that restores it.
Document verification	Upload PDF/image, get a labelled heuristic quality verdict, then an officer explicitly verifies or rejects with a reason.
Reports	Scope-aware, printable via the browser: summary, stage breakdown, stage-duration-vs-SLA chart, top 10 risk parcels, project progress.
Alerts	Notification bell aggregating stuck parcels, missing documents and open objections; each alert links straight to its parcel.
SMS previews	Simulated landowner notifications drafted in the owner's recorded language (Hindi / Marathi / English) at stage changes, objection updates and alerts.
Landowner portal	Zero-login survey-number search (typed or spoken), status with plain-language “action required”, stepper, documents, an illustrative compensation calculator, objection filing, and objection tracking.
Accessibility & last mile	Hindi/English UI toggle, voice input, text-to-speech read-aloud, a data-saver mode that drops map tiles, and a dark theme.

Running it — the operations manual

4.1 Getting it running

You need **Node.js 18 or newer** and nothing else. No database, no API keys, no Docker. From the project folder:

```
npm install      # once, installs dependencies
npm run dev      # starts the dev server, prints a http://localhost:5173 URL
npm run build    # type-checks (tsc -b) then produces a production build in dist/
npm run preview  # serves the built dist/ folder, to check the production build
```

REHEARSE THIS BEFORE THE DEMO

Run `npm run build` the night before. It runs the TypeScript compiler first, so if anything is broken you find out then and not in front of a judge. And have the dev server *already running* when you walk in — never open a terminal on stage.

4.2 Demo mode vs Supabase mode

The app decides which backend to use at startup, in `src/data/index.ts`, using one boolean:

```
export const repository = isSupabaseConfigured ? supabaseRepository : demoRepository;
```

And `isSupabaseConfigured` is simply whether both environment variables are set:

IF...	THEN...
<code>.env.local</code> is absent or empty	Demo mode. All 42 parcels come from <code>src/domain/demoData.ts</code> , held in memory. Edits (advancing a stage, uploading, filing an objection) persist while you navigate, and reset on a full page reload. No network at all.
<code>VITE_SUPABASE_URL</code> and <code>VITE_SUPABASE_ANON_KEY</code> are set	Live mode. The same screens read and write PostgreSQL through Supabase, and uploaded files go to the <code>parcel-documents</code> storage bucket. Run <code>supabase/schema.sql</code> once on a fresh project first.

WHY THIS MATTERS FOR THE PITCH

Demo mode means the app **cannot fail because the venue Wi-Fi is down** — a real risk at SIH. It also means you can honestly say “a district can pilot this before any procurement”: the same code, the same screens, one environment variable apart. Demo the offline mode; mention the live mode.

4.3 The route map

ROUTE	GUARD	PAGE
<code>/</code>	none	Landing page — the pitch, the stage walk, and the sign-in role picker
<code>/official</code>	4 official roles	District dashboard (national/state roles are redirected to the national view)
<code>/official/national</code>	National Admin, State Authority only	National dashboard

/official/parcel/:id	4 official roles + parcel must be in scope	Parcel workspace
/official/action-center	4 official roles	Risk-ranked queue
/official/reports	4 official roles	Printable report pack
/official/access-restricted	none (deliberately)	Access Restricted notice
/landowner	none	Survey-number search
/landowner/status/:id	none	Citizen status view

Note the deliberate asymmetry: `/official/access-restricted` sits *outside* the role guard. If it were inside, a role that isn't allowed would be redirected to it, and then redirected again, forever. That is a real bug we avoided by design, and it is a good answer if a judge asks about edge cases.

4.4 Screen by screen

Landing page — /

The public front. Animated hero, a seven-stage “stage walk”, capability cards that open detail modals, and at `#sign-in` the role picker. Picking District Officer or Field Officer reveals a **state** dropdown and then a **district** dropdown — both are populated from the actual data, so the district list changes with the state you choose. State Authority needs only a state. National Admin and Landowner need nothing.

District dashboard — /official

1. **Three summary tiles** — total parcels and how many complete; stuck parcels and how many blocked; missing documents and open objections.
2. **Count by stage** — seven tiles showing how many parcels sit in each stage right now.
3. **Filters** — survey number search, district, village, project, stage, status. All six combine.
4. **Map** — OpenStreetMap tiles, one circle marker per parcel, project boundaries drawn as convex hulls. Toggle between *colour by project* and *colour by status*. Each marker's popup links into the parcel.
5. **Attention queue** — the six worst parcels by days-in-stage, each with its blocking reason.
6. **Parcel list** — the full filtered table: survey, village, district, stage, days in stage, status, missing docs, open objections.

National dashboard — /official/national

Five summary cards (coverage, area, compensation, timeline status, social impact/R&R), then a Gantt-style **project timeline** with a “today” line and per-project progress fill, then the **R&R by project** table, then the **project-wise progress** table.

Parcel workspace — /official/parcel/:id

This is the demo centrepiece. Top to bottom: parcel overview (owner, phone, location, area, coordinates, compensation estimate) · workflow status (stage, days vs threshold, status badge, missing documents, open objections) · **risk assessment** (score, level, responsible role, all four contributors with their points, and the recommended action in plain English) · the seven-step stepper · **Advance Workflow** (either the form, or an explanation of exactly what is blocking) · the **audit chain ledger** · document upload · the document table with Verify / Reject · the objection table with a status dropdown and an SMS preview button per row.

Action Center — /official/action-center

One table, every in-scope parcel, sorted by risk score descending: survey, district, stage, score, level badge, the contributing reasons with their points, and the recommended action.

Reports — /official/reports

Summary table, stage breakdown, the **stage duration vs SLA chart**, top-10 risk parcels, and project progress. The **Print** button calls the browser's own print dialog — which is also how you produce a PDF.

Landowner portal — /landowner

A single search field with a **microphone button** next to it. Speaking “one two four slash seven” is normalised to **124/7** and searched automatically. The status page then shows: overview, status with *Action required* in plain language, the stepper, documents on file, an **illustrative** compensation calculator (area × rate × factor, explicitly labelled as not the official award), an objection form, and the objections you have already filed with their live status.

4.5 The global controls in the header

CONTROL	WHAT IT DOES
 Language	Toggles the whole UI between English and Hindi. Choice is remembered in <code>localStorage</code> .
 Data saver	When on, the map and the project timeline are replaced by a short explanatory panel — no tile downloads. For weak district networks.
☾ / 🌟 Theme	Light/dark. Sets <code>data-theme</code> on the document root.
 Alerts	Badge count of in-scope alerts. Opens a panel; each alert links to its parcel and carries its own SMS preview button. Hidden entirely for the landowner role.

4.6 Troubleshooting

SYMPTOM	CAUSE AND FIX
Signing in bounces you back to the landing page	No session. <code>RequireRole</code> redirects to <code>/</code> when there is no stored session — pick a role and complete the state/district dropdowns; the sign-in button stays disabled until the scope is complete.
“Access Restricted” on the national dashboard	Correct behaviour. Only National Admin and State Authority may open <code>/official/national</code> .
Parcel page says not found	Either a bad id, or the parcel is outside your district scope. Both are intentional.
The map is blank	Data saver is on, or the current filters match zero parcels.
Microphone button missing	The browser has no Web Speech API. Use Chrome or Edge. The button hides itself rather than showing a broken control.
Audit chain shows a plain table	<code>window.crypto.subtle</code> is unavailable — that happens on insecure origins. <code>localhost</code> counts as secure, so this only bites if you serve over plain HTTP from another machine.
Your edits vanished	You reloaded the page in demo mode. In-memory store resets on reload. Expected.

The demo script

Six minutes, three roles, one story: **the nation sees the problem** → **the officer is told exactly what to do** → **the citizen can see it too**. Every number below is what the app will actually show. If a number on screen differs from this page, trust the screen and tell the team.

BEFORE YOU START

Dev server already running · browser at `localhost:5173` on the landing page · zoom at 100% · data saver **off** · theme light · language English · **hard-reload once** so the in-memory data is pristine (a previous rehearsal may have advanced a parcel).

Act 1 — The national picture (90 seconds)

1. On the landing page, scroll to **Sign in**. Choose **National Admin**. No scope needed. Sign in.
2. You land on the **National Dashboard**. Read the cards aloud:
 - **5 projects across 5 states**
 - **8.74 ha acquired of 76.23 ha notified** — about 11%
 - **₹3,42,90,000 paid of ₹13,69,50,000 assessed** — 25%
 - **5 on track**, 0 at risk, 0 delayed
 - **197 of 371 displaced families resettled** (53%), out of 1,425 affected; **2 of 5** projects have a complete R&R checklist
3. Line to say: *"Nobody had to send in a return for this. It is computed from the same parcel records the district officer is editing right now."*
4. Point at the **project timeline** — the vertical line is today; the fill inside each bar is how far that project has actually progressed through the seven stages.
5. Scroll to the two tables. Note the R&R table has its own checklist column, because the problem statement asks for R&R progress as its own thing.

Act 2 — The officer's day (3 minutes — this is the important one)

1. Go back to `/` and sign in again as **District Officer**, state **Maharashtra**, district **Pune**.
2. You now see the **District Dashboard** with exactly **6 parcels**, of which **3 are stuck** and **1 document is missing**. Say: *"Same application, same code — this officer simply cannot see the other 36 parcels."*
3. Open the **Action Center** from the sidebar. Top row is **124/7**, score **55, High**, reasons **Stage delay (40), Missing documents (15)**, recommended action: *"Valuation Officer should verify the missing Valuation report document(s) to unblock this parcel."*
Line: *"It doesn't just tell him something is wrong. It names the officer and the document."*
4. Click **124/7** to open the parcel workspace. Walk down it:
 - Owner **Kavita Patil**, Dhanori village, Haveli tehsil, Pune, **1.42 ha**, estimate **₹36,80,000**.
 - Stage **Valuation, 57 days of a 21-day threshold** — badge reads **Stuck**.
 - Risk card: **55 / 100, High**, responsible **Valuation Officer**, contributors **Stage delay 40 · Missing documents 15 · Open objections 0 · Deadline proximity 0**.
Line: *"Every point is accounted for. There is no model here to argue with."*

5. **The gate.** Scroll to *Advance Workflow*. There is no button — instead: **"Missing required document: Valuation report."**
Line: *"This is the heart of it. The officer cannot mark this parcel as progressed. Not 'shouldn't' — cannot."*
6. **Unblock it live.** Scroll to *Upload document*. Stage **Valuation**, type **Valuation report**, choose any PDF, upload. A labelled quality verdict appears. Then in the documents table click **Verify** on that row.
Now scroll back up: the gate has opened and **Advance to Compensation Approval** is live. Click it.
Line: *"Document verified, gate re-evaluated, parcel moved — and watch what happened underneath."*
7. **The audit chain.** Scroll to the ledger. Every history entry now carries a SHA-256 hash that folds in the previous entry's hash. Badge reads **chain verified, N of N links intact**.
Click **Simulate tamper**. The first entry is edited — and the badge flips to broken, with that link and *every link after it* marked red.
Line: *"You cannot quietly back-date a land record here. Changing one entry breaks every entry after it, visibly."*
Then click **Reset**.
8. **The SMS.** In the Advance Workflow card, click **Notify Landowner (SMS)**. Kavita Patil's recorded language is Hindi, so the drafted message is in Hindi:
BhoomiSetu: आपकी भूमि (सर्वे 124/7) अब "मूल्यांकन" चरण में पहुँच गई है।
Say plainly: *"Simulated — no SMS gateway is wired in. The message composition and language selection are real."*

Act 3 — The citizen (90 seconds)

1. Click **Landowner** in the top nav. Note out loud: *"No sign-in. None."*
2. Press the **microphone** and say **"one two four slash seven"**. It normalises to **124/7** and searches automatically.
(Type it if the room is loud.)
3. On the status page, point at **Action required** — plain language, not jargon.
4. Press  **Listen** — the browser reads the status aloud. *"For someone who cannot read the screen."*
5. Toggle  **language** to Hindi. The whole interface switches.
6. Scroll to the **compensation calculator** — read the disclaimer aloud. *"It is labelled illustrative, because the statutory award is not ours to compute."*
7. File an objection: pick a reason, type a line, submit. It appears immediately in **Your objections** with a tracking id.
Closing line: *"And that objection is now on the officer's screen too — same record, both sides."*

IF YOU ONLY GET 90 SECONDS

Skip Acts 1 and 3. Open **124/7** directly, show the blocked gate, upload & verify the document, advance, and break the hash chain. That sequence alone carries the entire idea.

TWO THINGS THAT CAN GO WRONG ON STAGE

The parcel is already advanced. Somebody rehearsed and did not reload. Hard-reload the page — demo data resets to seed.

The microphone does nothing. Browser permission, or no Web Speech support. Type the survey number and move on; do not debug on stage. Mention that the button hides itself where the API is absent, which is deliberate.

The tech stack — every choice and why

For each one: what it is, why it is in the project, and **what we rejected instead**. The rejection is the part judges probe, because it separates “we picked the thing in the tutorial” from “we made a decision”.

6.1 React 19

What it is. A JavaScript library for building user interfaces out of components — self-contained pieces that take data in and describe what should be on screen. When the data changes, React works out the minimum change to make to the page.

Why here. This app is one dataset viewed six different ways. Change a parcel's stage and the dashboard tiles, the map marker colour, the alert count, the risk queue position and the stepper all have to change together. That is exactly the problem React exists for.

WHY NOT THE ALTERNATIVES

Plain HTML + JavaScript: we would hand-write the DOM updates for all six of those views, and they would drift out of sync — the classic source of “the dashboard says 13 but the table shows 12”. **Angular:** heavier, and its structure earns its keep on large teams, not four people in a hackathon. **Vue:** a genuinely reasonable choice; we picked React for the ecosystem depth (react-leaflet, Framer Motion, React Router) and because it is the most commonly read stack in evaluation.

6.2 TypeScript

What it is. JavaScript with types. You declare that a parcel has a `currentStage` which can only be one of seven exact strings, and the compiler refuses to build if you ever write an eighth.

Why here. This is a rules-heavy domain. There are seven stages, five official roles, eight document kinds, three objection statuses, four risk levels. The types make illegal states unrepresentable — you cannot accidentally set a parcel's stage to `"aproval"`. `npm run build` runs `tsc -b` first, so a typo is a build failure, not a runtime crash in front of a judge.

WHY NOT PLAIN JAVASCRIPT

Speed, supposedly. But in a domain with this many enumerated values, the time you save not writing types you lose twice over debugging a mistyped stage id at 3am. There is also a governance argument worth making to judges: *a department can read the type definitions and see the entire domain model in one file.*

6.3 Vite

What it is. The build tool and dev server. It serves your source files to the browser during development with near-instant reloads, and bundles an optimised build for production.

Why here. Startup and reload speed, which over a multi-day build is the single biggest quality-of-life factor. It also gives us `import.meta.env`, which is how the Supabase switch reads its configuration.

WHY NOT THE ALTERNATIVES

Create React App: deprecated and slow. **Webpack by hand:** a day of configuration we did not have. **Next.js:** excellent, but it is a server-side framework, and this app deliberately has no server of its own — it is a static bundle that talks to a data adapter. Choosing Next.js would have added a Node server to deploy for zero benefit.

6.4 React Router 7

What it is. Maps URLs to pages, and lets pages be nested inside shared layouts.

Why here. Two things we needed specifically: **nested layout routes**, so the app header and the official sidebar are declared once and every child page inherits them; and **route guards**, so `RequireRole` wraps a whole subtree of routes rather than each page checking permissions itself. Also: real, shareable URLs — `/official/parcel/parcel-124-7` is a link you can send to a colleague, which matters for a workflow tool.

6.5 Leaflet + react-leaflet + OpenStreetMap

What it is. Leaflet is an open-source mapping library; react-leaflet is its React binding; OpenStreetMap supplies the map tiles.

Why here. Land acquisition is inherently spatial — an officer thinks in “which parcels along this stretch”, not in table rows. We draw one marker per parcel and each project’s boundary as a convex hull of its parcels.

WHY NOT GOOGLE MAPS OR MAPBOX

Both need an API key and a billing account, and both meter usage. For a government system that is a procurement conversation and a data-sovereignty question. Leaflet plus OSM has **no key, no bill, no vendor**, and OSM tiles can be self-hosted inside a government network — which is the answer if a judge asks about deploying on NIC infrastructure.

6.6 Framer Motion

What it is. An animation library for React.

Why here. Only on the landing page — the reveal-on-scroll and the stage walk. It is presentation polish, and we are honest that it is. Importantly it respects `useReducedMotion()`, so a user who has asked their operating system to reduce motion gets a still page. That is an accessibility requirement, not a nicety.

6.7 Supabase (PostgreSQL + Storage)

What it is. A hosted PostgreSQL database with an auto-generated API and a file storage service.

Why here. We needed a real database path without spending the hackathon writing a backend. Supabase gives us Postgres — which is what a government deployment would actually run — plus file storage for scanned documents, reachable directly from the browser. Our schema mirrors the TypeScript domain: the seven stages are a Postgres `enum`, so an invalid stage is rejected by the database as well as by the compiler.

WHY NOT FIREBASE, OR OUR OWN EXPRESS + MONGODB

Firestore is document-based; our data is deeply relational (projects → parcels → history / documents / objections) and we want real foreign keys and enums. **Express + MongoDB** means writing, hosting and securing a server we do not have time to write, and MongoDB again loses the enum and foreign-key guarantees that make this domain safe. And critically: **Postgres is what states already run**, so the migration story is credible.

6.8 Web Crypto API — SHA-256

What it is. A cryptographic hashing function built into every modern browser via `window.crypto.subtle`. It turns any input into a fixed 64-character fingerprint; change one character of input and the fingerprint changes completely, and you cannot work backwards from the fingerprint to the input.

Why here. Our audit ledger hashes each stage-history entry *together with the previous entry's hash*. That chaining is what makes it tamper-evident: altering entry one changes its hash, which changes entry two's input, and so on down the chain. The “Simulate tamper” button demonstrates it in one click.

WHY NOT A BLOCKCHAIN

Say this exactly, because you will be asked. A blockchain solves the problem of *mutually distrusting parties with no shared authority*. Land acquisition has a clear authority — the Collector, the state, the Department. What we need is not consensus, it is **evidence of alteration**. A hash chain gives us exactly that at zero cost: no network, no tokens, no nodes, no energy, no procurement. Adding a blockchain would be architecture theatre.

Why not just a database audit table? Because whoever can edit the record can edit the audit table. The hash chain means an edit is detectable even by someone who does not trust the database administrator — and the verification runs client-side, with no server round-trip.

6.9 Web Speech API

What it is. Two browser features: speech *recognition* (microphone to text) and speech *synthesis* (text read aloud).

Why here. The landowner portal's users may have limited literacy. Voice input lets someone speak their survey number; read-aloud speaks their status back in Hindi or English. Both buttons detect support and hide themselves entirely when the browser lacks the API — a missing feature is better than a broken button.

WHY NOT A CLOUD SPEECH SERVICE

Google or Azure speech would be more accurate, but each carries an API key, a per-minute bill, and — the decisive one — **it sends a citizen's voice to a third-party server**. For a government citizen-facing portal that is a data-protection problem we should not create. The browser API runs where the user is.

6.10 What we deliberately did *not* use

Judges reward restraint if you can justify it. Each of these is a real decision.

NOT USED	WHY NOT
Any ML / AI model	The risk score must be explainable to the officer it flags and auditable by a department. Four weighted signals, all visible, beat an accurate black box in a governance setting. We also have no labelled training data for Indian land-acquisition delay — anything we trained would be invented.
A charting library (Chart.js, Recharts, D3)	We have exactly two charts — the stage-duration bars and the project timeline. Both are percentage-width <code></code> elements in CSS. Adding a charting library would have cost hundreds of kilobytes of bundle for two components, on an app that must work on weak district networks.
A UI component library (MUI, Ant, Bootstrap)	Government portals have their own look, and every one of these ships a strong opinion plus a large bundle. We wrote eleven small components in <code>src/components/ui.tsx</code> instead, which keeps the markup semantic and accessible and the CSS ours.
Tailwind CSS	Plain CSS with custom properties gives us theming (light/dark) in a handful of variables, and keeps the HTML readable for anyone auditing it later.
Redux / Zustand / TanStack Query	Our shared state is small: language, theme, data-saver, and the session. Four React contexts cover it. Page data is fetched per page and memoised with <code>useMemo</code> . Reaching for a state library here would be complexity without a problem.
Real authentication	Explicitly out of scope for the prototype — see Chapter 11. We are honest about this rather than shipping a fake login box that implies security we do not have.

Architecture

7.1 The five layers

Read it top-down: who is asking → what they see → the rules that decide → where the data comes from → where it rests.

LAYER	CONTENTS
1. Access	Five roles picked at the landing page. Session (role + optional state + district) stored in <code>localStorage</code> , guarded by <code>RequireRole</code> , scoped by <code>access.ts</code> .
2. Experience	Nine pages and about fifteen components. React 19, TypeScript, Vite, React Router, Leaflet, Framer Motion.
3. Domain rules engine	The stage gate, document requirements, risk scoring, scope filtering, the SHA-256 audit chain, SLA/timeline maths, SMS composition and convex-hull geometry. <i>Pure TypeScript: no network, no randomness, no React.</i>
4. Data access	One <code>ParcelRepository</code> interface with eleven methods. Two implementations: demo (in memory) and Supabase (live).
5. Persistence	PostgreSQL with enums mirroring the domain, plus an object-storage bucket for scanned documents.

THE SINGLE MOST IMPORTANT ARCHITECTURAL FACT

Layer 3 has **no imports from React and no network calls**. It is plain functions: same input, same output, every time. That is why we can say a department could audit the scoring, and why the whole engine could be lifted into a server or a batch job unchanged. If a judge asks “what is the interesting engineering decision here?”, this is the answer.

7.2 The repository pattern — and why it is the deployment story

Every page imports one thing for data:

```
import { repository } from '../data';
const parcels = await repository.listParcels();
```

No page knows or cares which backend is behind it. `src/data/index.ts` makes that decision once:

```
export const repository: ParcelRepository =
  isSupabaseConfigured ? supabaseRepository : demoRepository;
```

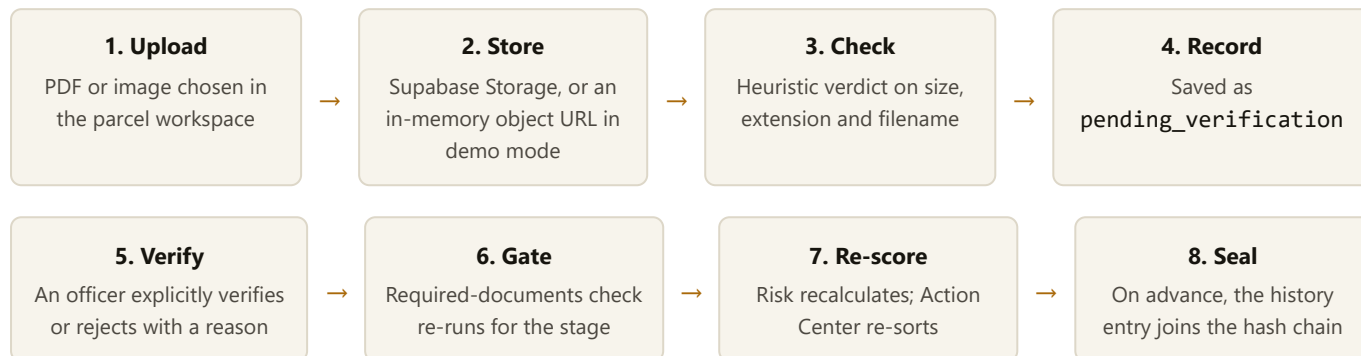
Both implementations satisfy the same TypeScript interface, so the compiler guarantees they are interchangeable. The interface is small and explicit:

```
listParcels()
advanceParcelStage(input)
addObjection(input)
getProjectById(id)
getParcelById(id)
addDocument(input)
updateObjectionStatus(i)
getParcelBySurveyNumber(n)
verifyDocument(input)
listProjects()
```

WHY THIS IS THE ANSWER TO “HOW WOULD A STATE ACTUALLY ADOPT THIS?”

A third implementation — call it `dilrmpRepository` — that talks to a state's existing land-records API would satisfy the same interface. Nothing above layer 4 would change: not a page, not a rule, not a test. **Integration becomes writing one adapter, not rewriting the product.** That is a genuinely strong answer and it is true of the code as written.

7.3 What happens when one document is uploaded



Step 5 is deliberately a human step. An uploaded document is *not* a satisfied requirement — only a *verified* one is. That single design decision is what stops the system from being gamed by uploading a blank page.

7.4 The file map

PATH	WHAT LIVES THERE
Domain — the rules engine	
<code>domain/constants.ts</code>	Stages, roles, document kinds, statuses, sectors, states, risk levels. The single source of truth for the whole domain.
<code>domain/types.ts</code>	Every entity shape: parcel, project, document, objection, history entry, and all computed result types.
<code>domain/rules.ts</code>	The gate, parcel status, dashboard and national summaries, alerts, stage-duration stats.
<code>domain/risk.ts</code>	The 0–100 score, its four contributors, the recommended-action text, the Action Center queue.
<code>domain/auditChain.ts</code>	SHA-256 chain build and verify.
<code>domain/access.ts</code>	State/district scoping of projects and parcels.
<code>domain/documentCheck.ts</code>	The labelled heuristic document quality check.
<code>domain/smsPreview.ts</code>	Message composition in the owner's language.
<code>domain/timeline.ts</code>	The project Gantt axis maths.
<code>domain/geo.ts</code>	Convex hull for project boundaries.
<code>domain/demoData.ts</code>	The 42 seeded parcels and 5 projects, generated from compact seeds.
Data — the boundary	
<code>data/types.ts</code>	The <code>ParcelRepository</code> interface and its input types.
<code>data/index.ts</code>	The one line that picks the backend.
<code>data/demoRepository.ts</code>	In-memory implementation. Immutable updates so React sees changes.
<code>data/supabaseRepository.ts</code>	PostgreSQL implementation.
<code>data/upload.ts</code>	File upload: Supabase Storage, or an object URL in demo mode.

Pages & components

<code>pages/</code>	Landing, District dashboard, National dashboard, Parcel detail, Action Center, Reports, Landowner search, Landowner status, Access restricted.
<code>components/ui.tsx</code>	The eleven-component in-house UI kit.
<code>components/</code>	AppShell, OfficialShell, RequireRole, ParcelMap, AuditChainLedger, NotificationCenter, SmsPreviewPanel, ProjectTimeline, StageDurationChart, SpeakButton, VoiceInputButton.
<code>i18n/</code>	Translations plus the Language, Session and DataSaver contexts.
<code>theme/</code>	Light/dark theme context.
<code>supabase/schema.sql</code>	Tables, enums, indexes, RLS statements and the storage bucket.

The domain model

Five entities. If you understand these, you understand the data behind every screen.

AcquisitionProject

FIELD	MEANING
name , sector , state	Sector is one of eight (highway, railway, irrigation, industrial corridor, power transmission, urban infrastructure, port, mining); state is one of ten.
implementingAgency	Who is executing it.
sanctionedOn , targetCompletionOn	The two dates that define the timeline bar and the deadline-proximity risk signal.
totalAreaRequiredHectares , compensationSanctioned	Budgeted totals.
rAndR	Affected families, displaced families, families resettled, and whether the R&R checklist is complete.

AcquisitionParcel — the centre of the system

FIELD	MEANING
surveyNumber	The human identifier, e.g. 124/7 . This is what a landowner searches by.
owner	Name, phone, and preferredLanguage (en / hi / mr) — the last one drives SMS language.
village , tehsil , district	Revenue hierarchy. District is what scoping filters on.
areaHectares	Feeds the area-acquired rollup.
currentStage , stageEnteredOn	The two fields the entire SLA and risk system is computed from.
compensationEstimate , compensationPaid	Assessed vs disbursed.
coordinates	lat/lng for the map marker and the project hull.
history[]	Stage history entries — the input to the audit chain and to stage-duration stats.
documents[]	Uploaded documents with their verification status.
objections[]	Objections filed against this parcel.

StageHistoryEntry

stage · enteredOn · exitedOn (absent while the stage is still open) · handledByRole · note . These five fields are exactly what the hash covers, so the chain protects who did what, when, and why.

ParcelDocument

stage · kind (one of eight) · title · uploadedOn / uploadedByRole · fileType · url · status (pending_verification / verified / rejected) · rejectionReason · reviewedByRole / reviewedOn · qualityCheckVerdict .

The one to remember: the gate counts a required document as present only when its status is **verified**.
Uploading is not satisfying.

ParcelObjection

submittedOn / **submittedBy** · **reason** (ownership dispute, measurement error, valuation objection, compensation dispute, other) · **description** · **status** (pending / under review / resolved) · **updatedOn** · **assignedToRole**. An objection counts as **open** if its status is anything other than **resolved**.

The rules engine, formula by formula

Everything in this chapter is deterministic. Nothing is random, nothing is trained, nothing calls the network. If you can explain these seven mechanisms you can answer any technical question about the product.

9.1 The advance gate — `getAdvanceGate()`

The single most important function in the codebase. It returns either “yes, and here is the next stage” or “no, and here is exactly why”.

A parcel may advance if and only if ALL of:

1. a next stage exists (not already at Possession)
2. every required document for the CURRENT stage exists AND is status 'verified'
3. if the current stage is Objection Review: no objection is open (an objection is open when status `!= 'resolved'`)

Otherwise it returns `canAdvance:false` plus a human-readable `reasons[]` list.

The subtlety worth pointing out to a judge: rule 3 only bites in Objection Review. An open objection elsewhere is *tracked and scored* but does not freeze the file — because in the real process an objection raised late does not automatically halt an award that has already been declared. Blocking everywhere would have been simpler and wrong.

9.2 Parcel status — `getParcelCalculatedStatus()`

Status is never stored. It is derived every time from the parcel plus today's date, which is why it can never be stale or wrong. Five values, evaluated in this exact precedence:

STATUS	CONDITION
Complete	No next stage exists <i>and</i> nothing is missing — the parcel is through Possession.
Stuck	<code>daysInStage > thresholdDays</code> for the current stage. <i>Checked before “blocked”, so a parcel that is both reports as stuck — the more urgent fact.</i>
Blocked	Within its SLA, but the gate refuses: a document is missing, or an objection is open in Objection Review.
Ready to advance	Within SLA and the gate is open. Someone just has to click.
On track	The fallback.

`daysInStage` is `daysBetween(stageEnteredOn, today)`, floored at zero, using UTC dates so no timezone can shift a day count.

9.3 The risk score — `getParcelRiskAssessment()`

Four signals, summed, clamped to 0–100. **Learn these four formulas exactly** — this is the question you are most likely to be asked, and being able to recite it is the difference between “we built a risk engine” and “we understand our risk engine”.

SIGNAL	MAX	FORMULA
--------	-----	---------

Stage delay	40	If stuck: $\min(40, 20 + \text{round}((\text{daysInStage} - \text{threshold}) \times 1.5))$ If not stuck: $\text{round}(\text{daysInStage} / \text{threshold} \times 15)$ Being over the threshold starts you at 20 and climbs 1.5 points a day. Below it you scale smoothly to 15.
Missing documents	30	$\min(30, \text{missingCount} \times 15)$ One missing document = 15, two or more = 30.
Open objections	20	$\min(20, \text{openCount} \times (\text{stage is Objection Review} ? 20 : 10))$ An objection during Objection Review is twice as serious — there it is actually blocking.
Deadline proximity	10	$\text{clamp}(0, 10, \text{round}(10 - \text{daysToTarget} / 18))$ Zero while the project deadline is far away; reaches 10 as it arrives or passes. Uses <i>signed</i> days, so overdue projects score the maximum.

WORKED EXAMPLE — PARCEL 124/7, THE ONE YOU WILL DEMO

Stuck in Valuation, 57 days, threshold 21	$\min(40, 20 + \text{round}(36 \times 1.5)) = \min(40, 74)$	40
1 missing verified document (Valuation report)	$\min(30, 1 \times 15)$	15
0 open objections (its one objection is resolved)	0	0
Project deadline is 491 days away	$\max(0, \text{round}(10 - 27.3))$	0
Total		55 — High

Bands: 0–24 Low 25–49 Medium 50–74 High 75–100 Critical

The **recommended action** is chosen by a fixed priority order, so it is always the most actionable one: missing document → stuck past threshold → objections blocking Objection Review → objections elsewhere → deadline within 30 days → “on track”. The officer named is looked up from `STAGE_HANDLER_ROLE`, which maps each stage to the role that owns it.

9.4 The audit chain — `buildAuditChain()` / `verifyAuditChain()`

Each history entry is flattened to a canonical string and hashed together with the previous hash:

```
canonical(entry) = id | parcelId | stage | enteredOn | exitedOn | handledByRole | note

hash[0] = SHA-256( "000...0" + "|" + canonical(entry[0]) )    ← 64 zeros = genesis
hash[n] = SHA-256( hash[n-1] + "|" + canonical(entry[n]) )
```

Verification recomputes every hash from the current data and compares against the hashes sealed earlier. Because each hash folds in the one before it, **altering entry *n* invalidates entry *n* and every entry after it** — which is precisely what the UI shows when you press *Simulate tamper*.

Support is feature-detected. Where `window.crypto.subtle` is unavailable the component falls back to a plain history table rather than failing.

9.5 Scoping — `access.ts`

```
national_admin    → every project, every parcel
state_authority   → projects where project.state === session.stateScope
district_officer  → that state, AND parcels where parcel.district === session.districtScope
field_officer     → identical to district_officer
landowner        → no official-side project or parcel at all
```

Applied in two places: list pages filter through `scopeParcelsToSession()`, and the parcel detail page calls `isParcelInScope()` before it will render — so a pasted URL for an out-of-scope parcel shows “not found”, not the parcel.

9.6 Project status — `getProjectCalculatedStatus()`

This one compares *work done* against *time elapsed*:

```
progressFraction = Σ(stage order of each parcel) / (parcelCount × 7)
timeFraction     = daysElapsedSinceSanction / totalTimelineDays (capped at 1.5)

complete  if progressFraction ≥ 1
delayed   if the target date has already passed
on_track  if progressFraction ≥ timeFraction - 0.10
at_risk   if progressFraction ≥ timeFraction - 0.25
delayed   otherwise
```

In other words: a project is on track if its physical progress is within ten percentage points of where the calendar says it should be, at risk within twenty-five, and delayed beyond that. Area acquired and possession percentage both use *parcels that reached the Possession stage* as the proxy for “done”, because that is the one stage that genuinely means finished.

9.7 The smaller mechanisms

FUNCTION	WHAT IT DOES
<code>getAlerts()</code>	Flattens three conditions into one notification list: stuck (high severity), missing document (medium), open objection (medium). High severity sorts first. It adds no new detection logic — it re-uses the same gate functions, so the bell can never disagree with the dashboard.
<code>getStageDurationStats()</code>	Average actual days per stage vs its SLA, using only completed history entries (ones with an <code>exitedOn</code>). The current, still-open stage is deliberately excluded — including it would understate the average for exactly the stages most parcels are sitting in.
<code>runDocumentQualityCheck()</code>	A pure heuristic, explicitly labelled in the UI as a prototype check. Flags empty or under-8KB files; asks for review on files under 25KB or over 15MB, on an extension that does not match the detected type, or on names containing <i>screenshot</i> , <i>whatsapp</i> , <i>untitled</i> , <i>temp</i> or <i>test</i> . Same file, same verdict, every time.
<code>buildSmsPreview()</code>	Composes the message in the <i>owner's</i> recorded language, not the officer's UI language. Hindi templates exist for stages, objection statuses and alert types; Marathi has no templates yet and falls back to English rather than showing untranslated text.
<code>computeConvexHull()</code>	Andrew's monotone chain, about forty lines, computing each project's boundary polygon from its parcel coordinates. No geospatial library added.
<code>getProjectTimelineAxis()</code>	Builds one shared time axis spanning every project's sanctioned→target window, quarter tick marks, a “today” position, and each project as a bar — all as plain percentages, so the rendering is CSS rather than a charting library.

The real numbers

Every figure here was computed by running the actual rules engine over the actual seeded dataset, as of the system's reference date of **27 August 2026**. Know the bold ones by heart: a judge asking “how many parcels?” and getting a confident exact answer is worth more than it should be.

42

parcels

5

projects

5

states, 21 districts

10.1 Portfolio-wide (what National Admin sees)

METRIC	VALUE	PARCELS BY STAGE	COUNT
Total parcels	42	Notification	7
Stuck (past SLA)	13	Joint Survey	6
Blocked (gate refuses)	5	Objection Review	7
Ready to advance	19	Valuation	7
Complete	5	Compensation Approval	5
Missing documents	2	Award	5
Open objections	5	Possession	5
Active alerts	21		
NATIONAL ROLLUP		VALUE	
Area acquired / notified		8.74 ha of 76.23 ha (11%)	
Compensation paid / assessed		₹3,42,90,000 of ₹13,69,50,000 (25%)	
Project timeline status		5 on track, 0 at risk, 0 delayed	
R&R — families resettled / displaced		197 of 371 (53%)	
R&R — total affected families		1,425	
R&R checklist complete		2 of 5 projects	

10.2 The five projects

PROJECT	STATE	SECTOR	PARCELS	PROGRESS	DAYS TO TARGET
Pune–Nagpur Expressway Land Corridor	Maharashtra	National Highway	25	54%	491
Ahmedabad–Bharuch Dedicated Freight Corridor	Gujarat	Railway	5	60%	582
Narmada Basin Irrigation Expansion	Madhya Pradesh	Irrigation	4	43%	307
Nalgonda–Karimnagar Power Transmission Line	Telangana	Power Transmission	4	54%	399
Paradip Port Expansion & Approach Corridor	Odisha	Port	4	57%	522

10.3 What each role sees (verify this live before the pitch)

SIGNED IN AS	PARCELS	PROJECTS	STUCK	ALERTS	TOP OF QUEUE
National Admin	42	5	13	21	124/7 (55), 91/6 (55)
State Authority — Maharashtra	25	1	9	13	124/7 (55), 91/6 (55)
District Officer — MH / Pune	6	1	3	4	124/7 (55), 109/5 (40)
District Officer — MH / Amravati	2	1	2	3	91/6 (55), 18/11 (40)

10.4 The two parcels you will actually talk about

124/7 — THE HERO PARCEL

Kavita Patil · Dhanori, Haveli, **Pune** · 1.42 ha · estimate ₹36,80,000 · owner's language **Hindi**

Stage **Valuation** since 1 July 2026 — **57 days against a 21-day threshold**. Four documents, one objection (resolved), four history entries.

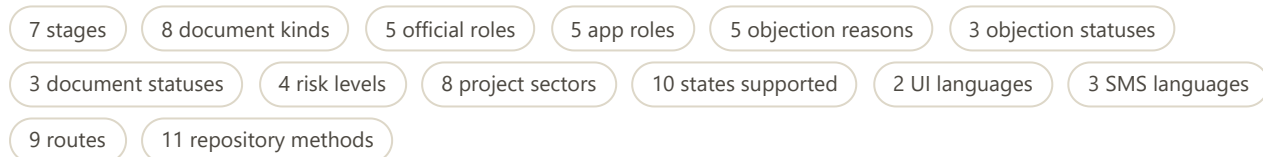
Risk **55 / 100 (High)**. Blocked by: *Missing required document: Valuation report*. Responsible: **Valuation Officer**.

91/6 — THE OTHER KIND OF BLOCK

Amravati district, stage **Objection Review**, **40 days against a 30-day threshold**, **1 open objection**.

Risk **55 / 100 (High)** — but from a different mix: stage delay 35 + open objections 20. Gate reason: *"Pending or under-review objections must be resolved before advancing."* Use this one if a judge asks whether documents are the only thing that can block a parcel.

10.5 System scale



Honest limitations

Read this chapter twice. Everything here is something a sharp judge can find in ten minutes. If you name it before they do, you look rigorous. If they find it first, everything else you said becomes suspect. The right posture is always the same: *“That part is simulated. Here is exactly what replaces it, and here is why we scoped it out.”*

11.1 There is no authentication

What is true: the landing page has a role *picker*, not a login. Anyone can select National Admin. The session is a role and an optional state/district kept in `localStorage`.

What is also true, and worth saying: the *authorisation* logic behind it is real and worth showing — role-based route guards, and data-layer scoping that blocks a pasted URL for an out-of-scope parcel. The missing piece is proving *who you are*, not deciding *what you may see*.

Say: “Authentication was explicitly out of scope for the prototype so we could build the workflow depth instead. In deployment this would be departmental SSO — the role and jurisdiction would come from the identity provider rather than a dropdown, and the scoping code we already have would consume it unchanged.”

11.2 The database policies are open, on purpose

THE CORRECTION MOST LIKELY TO CATCH YOU OUT

`supabase/schema.sql` enables row-level security on all five tables, but every policy is `using (true)` — that is, permissive. The file says so in a comment. Because the prototype has no authentication, both portals read and write through the same anonymous key, so restrictive policies would simply lock everyone out.

Therefore: do not say “access control is enforced by the database”. Today it is enforced in the application layer. Say instead: *“Row-level security is switched on and the policies are deliberately permissive because we have no authentication yet. The scoping rules are written and enforced in the data-access layer, and they are exactly the predicates that become RLS policies once identities exist — state and district on the JWT.”* That is accurate, and it shows you know what the production shape is.

11.3 The data is fictional

All 42 parcels, 5 projects, owner names, phone numbers, coordinates and amounts are invented, generated from compact seeds in `demoData.ts`. The *structure* mirrors real land records; the *content* does not correspond to any real parcel or person. Say so plainly — no team gets penalised for using synthetic data, and several get penalised for implying it is real.

11.4 The stage-duration chart currently shows a flat 15 days

Notice this before a judge does. The seeded history spaces every completed stage 15 days apart, so the Reports chart reports an average of 15 days for all six stages that have completed entries (sample sizes 35, 29, 22, 15, 10, 5; Possession has none yet, since no parcel has exited it).

Say: “The chart mechanism is real — it averages only completed history entries and compares against each stage's SLA. What is uniform is the seed data, because we generated the history procedurally. On real history you would

see the variance, which is the entire point of the chart.”

11.5 The document check is a heuristic, not AI

It inspects file size, extension-versus-detected-type, and filename patterns. It cannot read the document, verify a signature or confirm a seal. The UI labels it as a prototype check and the source file carries a comment saying it must never be presented as an authenticity check.

Say: “It catches the blank scan and the WhatsApp screenshot, which is most of what actually goes wrong at intake. It never decides anything — an officer still has to press Verify. The genuine document-authenticity step would be OCR plus a signature check, and that is deliberately future work.”

11.6 SMS is composed, not sent

No gateway is wired in and no network call is made. The message text, and the choice of language from the owner's record, are real; the delivery is not. The panel is labelled “simulated, not a real message” in the interface.

Say: “Wiring a gateway is a day's work and a procurement decision, not an architectural one. What we built is the part that has to be right: the trigger points and the language selection.”

11.7 Marathi is partial

The UI toggles between English and Hindi only. SMS templates exist in English and Hindi; an owner whose recorded language is Marathi receives the English template. This is a deliberate fallback rather than showing an untranslated key — but it is a gap, and the honest framing is “the mechanism handles N languages; we have written two.”

11.8 Demo-mode edits are in memory

Advancing a stage, uploading, verifying and filing objections all persist across navigation but reset on a full page reload. In Supabase mode they persist properly. This is the same repository interface either way — which is the point.

11.9 Everything is relative to a fixed date

The system computes “today” as the constant DEMO_REFERENCE_DATE = 2026-08-27, not the real clock. This is why the demo numbers are stable and reproducible — 124/7 is always 57 days into Valuation. In production this becomes the actual date. If a judge asks why the dates are in the future, that is the answer, and it is a good one: a demo whose numbers change under you is a demo you cannot rehearse.

11.10 What we would build next

NEXT	WHY IT IS NEXT
Departmental SSO + restrictive RLS	Turns the scoping we already compute into enforcement the database itself guarantees.
An adapter for a live land-records API	The single highest-value integration; the interface is already there waiting for it.
A real SMS/WhatsApp gateway	Closes the loop to the citizen who has no smartphone.
OCR and signature verification on documents	Upgrades intake from “plausible file” to “genuine document”.
Server-side scheduled escalation	Today an alert appears when someone opens the app; it should reach an inbox on the day it fires.

Full Marathi and more state languages	The translation layer is built; it needs content.
---------------------------------------	---

The judge Q&A bank

Forty questions, grouped. Practise these out loud, not in your head — the answers are short on purpose. If you do not know something, say “I don't know, but here is how I would find out”. That answer has never lost a hackathon; bluffing has.

On the problem and the domain

1. Why does land acquisition take so long?

Seven statutory stages, each with a different officer, different paperwork and its own hearing rights — held in separate registers. Nothing is wrong with any single step; the failure is that no one holds the whole chain, so slippage is invisible until a project review.

2. Which Act are you implementing?

RFCTLARR 2013. Our stages follow it: Section 11 notification, joint survey, Section 15 objections, valuation, compensation approval, Section 23 award, Section 38 possession. R&R entitlements come from the Second Schedule.

3. Isn't Bhoomi Rashi already doing this?

Bhoomi Rashi handles e-acquisition for National Highways under MoRTH, with compensation through PFMS. It is the closest existing system — and it is exactly why our design is adapter-based rather than a parallel silo. We are not proposing a replacement for state records; we are proposing the monitoring and decision-support layer above them.

4. What about DILRMP?

DILRMP is the record-digitisation programme. It produces the records; we consume them. Our repository interface is the seam where a DILRMP-backed adapter would plug in.

5. Who is your actual user?

Three, and they see different things: the district officer whose queue this reorders, the state or ministry official who no longer has to call for a return, and the landowner who today has no way to check anything.

6. Why does the landowner portal need no login?

Because a login is the barrier. The people most affected are least likely to complete an account creation flow. A survey number is already public information in the notification, and we show only citizen-facing fields.

7. Isn't that a privacy problem?

It is the real trade-off in the project, and worth stating as one. The status page shows stage, documents on file and objections — not identity documents or bank details. If a department wanted it gated, an OTP on the registered phone number is the obvious middle path, and it would be one screen.

On the workflow and the gate

8. What actually stops a parcel from advancing?

Two conditions. Every required document for the current stage must exist *and* be marked verified; and in Objection Review, no objection may be open. Otherwise the button is not there and the reason is printed instead.

9. Why does an open objection only block Objection Review?

Because in the real process an objection raised after an award has been declared does not automatically freeze the file. Blocking everywhere would be simpler and would not match practice. It still scores into risk everywhere.

10. Can an officer override the gate?

Not today. That is a deliberate prototype choice — it makes the rule demonstrable. A production system would need a recorded override with a reason and an authority level, and that override would itself go into the hash chain.

11. Why isn't uploading a document enough?

Because then the system could be satisfied with a blank page. Upload creates a pending record; a named officer must verify or reject it with a reason. Verification is the human step we refuse to automate.

12. Where do the day thresholds come from?

They are configured per stage in one constants file — 30, 21, 30, 21, 21, 21, 30. They are our working values for the prototype, and they are one edit away from whatever a state's own service standard says.

On the risk engine

13. Is this machine learning?

No, and deliberately not. It is four weighted signals summed to 0–100: stage delay up to 40, missing documents up to 30, open objections up to 20, deadline proximity up to 10. Every parcel shows its own breakdown.

14. Then why is it better than a sort by days?

Because days alone cannot distinguish a parcel that is late but unblocked from one that is late, missing a document and carrying an objection on a project that is nearly due. The score combines them, and the recommended action tells the officer which of those four things to fix first.

15. Where did the weights come from?

Reasoned, not fitted — we have no labelled delay data for Indian land acquisition, and inventing a training set would be worse than being explicit. Stage delay is weighted heaviest because it is the only signal that is always present and always worsening. The weights are constants in one file, and calibrating them against a state's historical data is the obvious first improvement.

16. Isn't "not AI" a weakness for a Smart Automation theme?

Automation is about removing manual work, and we remove a lot: no compiled returns, no manual sorting of a stuck-parcel list, no chasing which document is missing. What we did not do is add an unexplainable model to a decision that affects someone's land. An officer can recompute our score with a pen.

17. Walk me through one score.

124/7: stuck 57 days in Valuation against a 21-day threshold, so $20 + \text{round}(36 \times 1.5)$ capped at 40. One missing verified document, 15. No open objection, 0. Project due in 491 days, 0. Total 55, band High. On screen, in that breakdown.

18. What if everything scores high?

Then the ranking still works — it is a queue, not a threshold. The bands are for language; the sort is what an officer uses.

On the audit chain

19. Why not a blockchain?

Blockchains solve consensus between parties with no shared authority. Land acquisition has an authority. Our problem is evidence of alteration, and a hash chain gives exactly that — with no network, no nodes and no procurement.

20. Why not a database audit table?

Whoever can edit the record can edit the audit table. With chaining, altering one entry breaks that entry and every entry after it, and the verification runs in the browser without trusting the server.

21. Where are the hashes stored?

In the prototype the chain is sealed in the browser when the ledger loads, which is enough to demonstrate the property. Production would persist each hash with its entry, and ideally publish periodic checkpoints so even a full-database rewrite is detectable.

22. So it prevents tampering?

No — it *detects* it. That is the honest word and the correct one. Prevention is access control; this is evidence.

On architecture and technology

23. Why React?

One dataset, six synchronised views. Change a stage and the tiles, the map colour, the alert count, the queue position and the stepper must all agree. That is what React's model is for.

24. Why TypeScript?

Seven stages, five roles, eight document kinds, four risk levels. Types make an invalid stage impossible to write, and the build type-checks before it bundles. It also means a department can read the domain model in one file.

25. Why Leaflet and not Google Maps?

No API key, no billing, no vendor, and OSM tiles can be self-hosted inside a government network. For a public system that is a data-sovereignty answer, not just a cost one.

26. Why Supabase and not your own backend?

We wanted a real PostgreSQL path without spending the hackathon writing and securing a server. Postgres is what states already run, so the migration story is credible, and our schema mirrors the domain — the seven stages are a database enum.

27. How would a state actually adopt this?

Write one more implementation of our repository interface that talks to their records system. Nothing above the data layer changes — not a page, not a rule. Integration is an adapter, not a rewrite.

28. Can it scale to lakhs of parcels?

Not as it stands — every page loads all in-scope parcels and computes in the browser, which is right for 42 and wrong for 200,000. The fix is known and unglamorous: paginate the queries, and move the summaries into SQL views or a scheduled job. The rules engine is pure functions, so it lifts to the server unchanged. That property is why the answer is straightforward.

29. What's your test coverage?

We have no automated test suite — that is a real gap. What we do have is a rules layer written as pure functions with no network and no randomness, which is the precondition for testing it. Verification so far has been done by running each step in the browser and checking the figures against the engine.

30. Why no charting library?

Two charts. Both are percentage-width elements in CSS. A charting library would have added hundreds of kilobytes to an app that has to work on a weak district connection.

31. Does it work on a phone?

The layout is responsive and the landowner portal is the part most likely to be used on one. It has not been through a formal device-lab pass — say that rather than overclaiming.

32. Offline?

Demo mode runs entirely without a network, which is how we demo. That is not the same as offline-first field capture with sync — that would need a service worker and a conflict-resolution strategy, and we have not built it.

On security, data and governance

33. How do you handle authentication?

We don't — it was explicitly scoped out. The role picker is a demo device. The authorisation logic behind it is real: route guards plus data-layer scoping that blocks even a pasted URL.

34. Is the database secured?

Row-level security is enabled on every table, but the policies are deliberately permissive, because with no authentication both portals use the same anonymous key. Our scoping predicates are written in the application layer and are exactly what becomes RLS policy once identities exist.

35. Where do uploaded documents live?

A Supabase Storage bucket in live mode; an in-memory object URL in demo mode. In production this would be departmental storage with signed, time-limited URLs — the bucket is public in the prototype for demo convenience, which we would not ship.

36. Is the data real?

No. All 42 parcels, owners and amounts are synthetic. The structure mirrors real land records; the content is invented.

37. What about data protection / DPDP?

The relevant question is the landowner portal, since it shows parcel status without a login. Our position: survey numbers and acquisition notifications are already public, and we expose no identity or financial documents. If a department disagreed, an OTP gate on the registered number is one screen.

The uncomfortable ones

38. What's the hardest problem you solved?

Deciding what the gate should *not* block. It is easy to block everything and end up with a system officers route around. Objections block only in Objection Review; unverified documents block everywhere. Getting that boundary right took longer than writing it.

39. What would you do differently?

Write the rules engine's tests as we went. It is the one part of the codebase that is pure and deterministic, which makes it the cheapest thing in the project to test — and we didn't.

40. Why should this win?

Because it is not a dashboard over a status field. The gate makes the rule real, the score is explainable to the officer it flags, the ledger makes tampering visible, the landowner needs no login, and the whole thing runs unchanged against in-memory data or PostgreSQL — so a district can pilot it on Monday.

Pitch scripts

Three lengths. Learn the 30-second one word for word — it is what you say when a judge walks up mid-demo and asks “so what is this?”

30 seconds

“Land acquisition runs through seven statutory stages, and each one is tracked in a different register by a different officer. Nobody holds the whole chain for one parcel, so delay is discovered at a project review instead of on the day it starts.

BhoomiSetu puts all seven stages on one shared record. A parcel cannot advance unless that stage's documents are verified and no objection is open — enforced in code. Every parcel carries an explainable 0-to-100 delay risk score that names the responsible officer and the blocking document. And the landowner can check their own survey number with no login, in their own language.

It is running now, over 42 parcels across 5 projects and 5 states.”

2 minutes — the structure

TIME	BEAT	SAY ROUGHLY
0:00–0:20	The gap	Seven stages, seven registers. The Ministry has to ask states for a return. The landowner has no way to check anything. Delay is found after it happens.
0:20–0:35	The idea	One shared parcel record. Two portals. Seven gated stages.
0:35–1:05	The gate (show it)	Open 124/7. Stuck 57 days of 21. No advance button — “Missing required document: Valuation report.” Upload, verify, the gate opens, advance.
1:05–1:25	The intelligence	Risk 55 of 100, High. Stage delay 40, missing documents 15. Names the Valuation Officer. No model, no black box — an officer can recompute it by hand.
1:25–1:40	The integrity (show it)	Press Simulate tamper. That link and every link after it break. You cannot quietly back-date a land record here.
1:40–2:00	The citizen & the close	Landowner portal, no login, voice search, read-aloud, Hindi. Close on: one interface, two adapters — a district can pilot this before anything is procured.

5 minutes — add these

- **Open on the national dashboard** instead of the parcel: 5 projects, 5 states, ₹3.43 crore paid of ₹13.70 crore assessed, 197 of 371 families resettled — then say “computed live, no returns requested.”
- **Show the scoping:** sign in as District Officer for Pune, 6 parcels, then paste an Amravati parcel URL and get Access Restricted.
- **Show the Action Center** as a queue, not a search box.
- **Show Reports** and press Print — “this replaces the manual compilation.”
- **Name one limitation unprompted** — authentication is out of scope, and here is what replaces it. Judges remember the team that did this.

DELIVERY NOTES

- **Show, don't narrate.** The blocked gate and the broken hash chain are the two moments that land. Get to them fast.
- **Use the real numbers.** "57 days against a 21-day threshold" is persuasive; "it's been stuck a while" is not.
- **Never say "AI" loosely.** Say deterministic, explainable, auditable. Make it the boast.
- **One person drives, one person talks.** Do not narrate your own clicking.
- **When interrupted, answer and return.** "Good question — [answer]. Let me show you the piece that depends on it."

Who prepares what

Everyone learns Chapter 1, Chapter 10 and Chapter 11. Beyond that, split it — but the rule is that each person must be able to give a *two-sentence* version of everyone else's area. Judges test for the team that has one expert and three passengers.

ROLE ON THE DAY	OWNS	MUST BE ABLE TO ANSWER
The driver runs the laptop	Chapters 4 and 5	Every screen, every control, the demo path without looking. Recovers from a bad state without commentary. Knows the exact figures each role sees.
The narrator does the talking	Chapters 2 and 13	RFCTLARR, the seven stages, why acquisition stalls, the 30-second and 2-minute scripts. Questions 1–12 of the Q&A.
The engineer	Chapters 6, 7 and 9	Every technology choice and its rejected alternative. The four risk formulas from memory. The repository pattern as the deployment story. Questions 13–32.
The honest one	Chapters 11 and 12	Every limitation and its production answer — especially the RLS point in 11.2 and the flat 15-day chart in 11.4. Questions 33–40. This is the most valuable role and the least obvious one.

A rehearsal plan that actually works

1. **Run the demo end to end three times** on the machine you will use, with the projector if you can get one. Time it.
2. **Quiz round.** Each person picks ten questions from Chapter 12 and asks someone else — never the owner of that section. Any answer that takes more than 30 seconds needs shortening.
3. **Break it deliberately.** Advance the hero parcel, then recover. Turn on data saver mid-demo. Deny the microphone permission. You want the first time you see each failure to be in rehearsal.
4. **One hostile round.** Someone plays a sceptical judge and pushes on: is this just a dashboard, is the score meaningful, is anything secure. Practise not getting defensive.
5. **The night before:** `npm run build`, hard-reload the app, confirm the numbers in Chapter 10 still match. Charge the laptop.

Glossary

Domain terms

RFCTLARR	Right to Fair Compensation and Transparency in Land Acquisition, Rehabilitation and Resettlement Act, 2013. The statute the workflow follows.
Parcel	One piece of land being acquired, identified by survey number.
Survey number	The revenue identifier for a parcel, e.g. 124/7. What a landowner searches by.
Tehsil	Administrative unit below a district. Village → tehsil → district → state.
Section 11	Preliminary notification of intent to acquire — stage one.
Section 15	The landowner's right to object — stage three.
Section 23	The formal award declaring compensation — stage six.
Section 38	Taking physical possession — stage seven.
Award	The formal determination of compensation payable. Not a prize.
R&R	Rehabilitation and Resettlement — entitlements for displaced families, tracked separately from compensation.
DoLR	Department of Land Resources, under the Ministry of Rural Development. The problem statement's owner.
DILRMP	Digital India Land Records Modernisation Programme.
NGDRS	National Generic Document Registration System.
Bhoomi Rashi	MoRTH's e-land-acquisition portal for National Highways.
PFMS	Public Financial Management System — routes government payments.
Hectare	10,000 m ² . About 2.47 acres.

Technical terms

SLA	Service level agreement — here, the day threshold for a stage. Exceed it and the parcel is “stuck”.
The gate	Our name for the rule that decides whether a parcel may advance.
SHA-256	A hash function: any input becomes a fixed 64-character fingerprint. Changing the input changes the fingerprint completely.
Hash chain	Each entry's hash includes the previous entry's hash, so altering one entry breaks every later one.
Deterministic	Same input, same output, every time. No randomness, no model.
Pure function	Depends only on its arguments and changes nothing outside itself. Our whole rules layer.
Repository pattern	Putting data access behind an interface so the rest of the app doesn't know which backend it is talking to.
Adapter	One implementation of that interface. We have two; a state's system would be a third.
RLS	Row-level security — PostgreSQL deciding per row who may read or write. Enabled here, with deliberately permissive policies. See 11.2.
Enum	A type restricted to a fixed set of values. Our seven stages are one, in TypeScript and in Postgres.
Convex hull	The smallest polygon containing a set of points. How we draw a project boundary from its parcels.
Route guard	A wrapper that checks permission before a page renders. Ours is <code>RequireRole</code> .
localStorage	Browser-local key-value storage. Holds the session, language, theme and data-saver preference.
Web Speech API	Browser speech recognition and synthesis. Powers voice search and read-aloud, with no cloud service.
Web Crypto	Browser cryptography. Gives us SHA-256 with no library.

One last thing

The strongest sentence you have is not about technology. It is this: *“Every figure we have shown you — the risk weights, the SLA thresholds, the document lists, the hash chain — is read out of running code, not estimated for the deck.”* Say it, and then be able to prove it. That is what this handbook is for.